

System Architecture

1. Overview

This system is a **microservices-based e-commerce platform** for mechanical keyboard products.

It supports:

- Standard product purchasing
- Group-buy (demand-driven campaigns)
- Real-time updates using Streaming RPC

2. Technology Stack

Layer	Technology
Framework	Urunuat
API Gateway	Nginx
Communication	REST, gRPC, gRPC Streaming
Messaging	RabbitMQ
Authentication	JWT
Database Pattern	Database per service

3. Service Architecture

3.1 User Service (REST)

Responsibilities:

- User authentication (login/register)
- JWT token issuance
- Profile & address management

Database:

- users
- user_profiles

- addresses
-

3.2 Catalog Service (REST)

Responsibilities:

- Product & category management
- Product browsing

Database:

- categories
 - products
 - product_attributes
-

3.3 Inventory Service (gRPC + Streaming)

Responsibilities:

- Stock management
- Group-buy campaign management
- Real-time updates

Database:

- inventory
 - inventory_logs
 - group_buys
-

Communication Types

Unary gRPC:

- Used by Order Service
- Check stock
- Reserve stock

Streaming gRPC (Core Feature):

Used for real-time updates:

1. **Stock updates**
2. **Group-buy progress**

Streaming Use Cases

1. Live Stock Tracking

Client subscribes:

`SubscribeStock(product_id)`

Server streams:

100 → 95 → 80 → SOLD OUT

2. Group-Buy Progress (Primary Use Case)

Client subscribes:

`SubscribeGroupBuy(product_id)`

Server streams:

45 / 100

60 / 100

80 / 100

100 / 100 → SUCCESS

Streaming Triggers

Streaming updates occur when:

- Order is created
- Order is cancelled

- Inventory is updated
 - Group-buy participation changes
-

3.4 Order Service (REST + Event Producer)

Responsibilities:

- Order creation
- Checkout processing

Database:

- orders
 - order_items
 - payments
-

Communication

- Calls Inventory Service via gRPC (Unary)
 - Publishes events to RabbitMQ
-

Event Flow

Example:

```
{  
  "event": "ORDER_CREATED",  
  "orderId": "uuid",  
  "items": [...]  
}
```

3.5 Cart Service (REST)

Responsibilities:

- Manage shopping cart

Database:

- cart_items
-

4. Infrastructure Components

4.1 API Gateway (Nginx)

- Single entry point
 - Request routing
 - JWT validation (or forwarding)
-

4.2 Message Queue (RabbitMQ)

- Asynchronous communication
 - Decouples services
-

5. Core Business Flows

Normal Order Flow

1. User places order
 2. Order Service processes request
 3. Calls Inventory Service (gRPC)
 4. Inventory updates stock
 5. Event published
 6. Streaming updates stock
-

Group-Buy Flow

1. Admin creates group-buy campaign

2. Users place orders
 3. System updates group_buys.current_quantity
 4. Streaming pushes progress in real-time
 5. At deadline:
 - If target reached → SUCCESS → proceed production
 - Else → FAILED → cancel orders
-

6. Communication Patterns

Type	Usage
REST	Client → Services
gRPC	Internal service calls
gRPC Streaming	Real-time updates
Event-driven	Async processing

7. Authentication

JWT Flow

Client sends:

Authorization: Bearer <JWT_TOKEN>

Flow:

1. User logs in → receives token
 2. Token included in requests
 3. Gateway/services validate token
-

8. Data Management

- Database per service pattern
- No cross-service joins

- Event-based consistency
-